ETH zürich

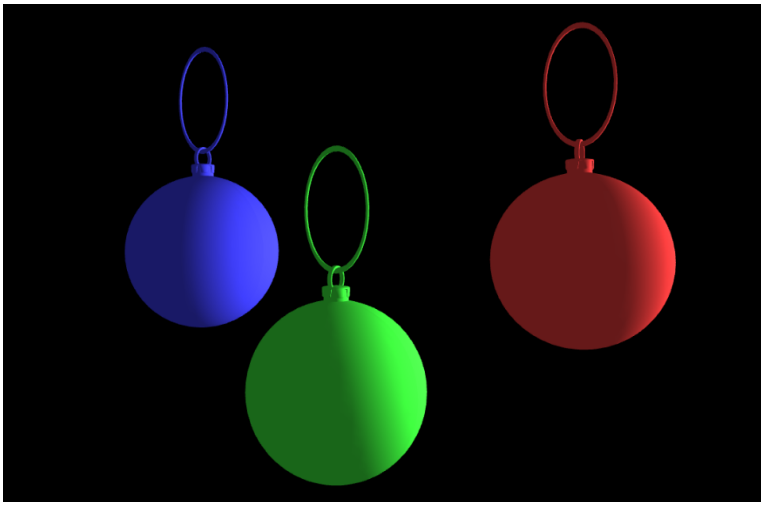
computer graphics laboratory

Prof. Dr. Markus Gross

Visual Computing

Exercise 7 / Week 8: Introduction to WebGL

Hand-out Date: 02 November 2026



Goals

- Getting acquainted with the WebGL graphics library API.
- Learning about the GL graphics pipeline, shaders, and buffers.
- Understanding how to apply colors and simple transformations.

Resources

Chapter 22 of the script (The Web Graphics Library) covers what you need for this exercise. Section 22.2.4 explains vertex buffers, attributes, indexed drawing and `gl.drawElements` (Tasks 1 and 3a). Transformations for Task 3b and the gl-matrix vector functions for Task 2 are in Section 22.2.5. Section 22.2.6 shows how to activate the shader program with `gl.useProgram`, set uniforms (Task 1b) and compute vertex normals from face normals (Task 2).

Read `readme.md` in the exercise folder before you start. It explains how to open the exercise through a local web server and how to find errors in the browser console. It also includes hints and spoilers for each task.

ETH zürich

computer graphics laboratory

Prof. Dr. Markus Gross

Tasks

1. Mesh Setup and Initialization

In this exercise, we will set up a mesh (Christmas decoration) for rendering. All code needed for this project is contained in `main.js` and the shader files `vertexShader.js` and `fragmentShader.js`.
First, get acquainted with the program structure. Then, have a look at the shader code. Note which attributes and uniforms are used in the fragment and vertex shader. Once you have familiarized yourself with the code structure, complete the following two tasks:

- **Binding the Data**: Locate the part of the code marked as "Task 1a". Insert code to create buffers for all the necessary vertex attributes (i.e., vertex positions and normals) and an index buffer for the mesh faces. Next, identify the per-vertex attribute inputs of the vertex shader and bind them to the correct array pointer (the procedure is covered in Section 22.2.4 of the script). Nothing is drawn yet, so the canvas stays black. The browser console should show no errors.
- **Uniform Variables**: Locate the part of the code marked as "Task 1b". Identify the uniform variable inputs of the vertex shader and bind them to the correct pointer (the procedure is covered in Section 22.2.6 of the script). Finally, draw the mesh using `gl.drawElements(...)`. When implemented correctly, the mesh will be displayed on the screen as a flat, very dark grey. The normals are still zero at this point, so the mesh only gets ambient light.

2. Normals for Lighting

In this part, we will add simple lighting in order to see the details of the 3D mesh. In the vertex shader, a simple diffuse lighting method is already implemented. This code, however, requires per-vertex normals to calculate the lighting. A triangle mesh only has well-defined normals per face, so we approximate the vertex normals from the face normals as follows (see code marked as "Task 2"). The OBJ file already contains vertex normals, but the code discards them so that you compute your own.

- **Face normals**: First, calculate the face normal for each triangle based on the vertex positions that correspond to each face by following the steps given as comments in the code (the gl-matrix vector functions are covered in Section 22.2.5 of the script).
- **Vertex Normals**: Next, approximate the vertex normals by averaging all the face normals around that vertex (the procedure is covered in Section 22.2.6 of the script).

- **Normalization**: In a final step, we iterate through the calculated vertex normals and normalize them to unit length. When implemented correctly, the baubles are still grey, but now shaded, with the light coming from the right.

3. Color and Rotation

In this part, we will color the mesh and rotate it. Note that the vertex color is currently set to `vec4(0.3, 0.3, 0.3, 1.0)` in the vertex shader. We want to extend the vertex shader to take a custom vertex color as attribute, which should be precomputed in the main code as follows:

- **Coloring**: In the code marked as "Task 3a", add a new array `mesh.vertexColors` to the mesh for storing the colors. Each color is a `vec4` (r, g, b, a), so `mesh.vertexColors` needs 4 numbers per vertex, one more than the positions and normals. Instead of giving the same color to all the Christmas baubles, color each of them differently (Hint: the mesh can be split up into three regions that separate the baubles using two planes going through $y = -8$ and $y = 9$). Then, add a new attribute `aVertexColor` to the vertex shader (also marked as "Task 3a" there) and extend the code in "Task 1a" to create a buffer for the new vertex colors and bind them analogously to the other attributes used in the vertex shader (see the procedure in 1a and Section 22.2.4 of the script). Finally, do not forget to use the new `aVertexColor` instead of the constant color inside the vertex shader. If implemented correctly, the three baubles will appear in different colors, similar to the image on the first page.
- **Rotation**: In order to rotate the mesh, apply a rotation around the z-axis to the `modelViewMatrix` in the code marked as "Task 3b" (rotations are covered in Section 22.2.5 of the script). When implemented correctly, the baubles turn around their vertical axis, and the light stays on the right.